

Hilos en una aplicación real

Dónde ya los estás usando, sin haberlo pedido

Este apartado es distinto

No vas a escribir código de producción — vas a aprender a ver algo que ya está pasando delante de ti sin que lo hayas notado.

Ahora mismo, mientras tu aplicación Spring Boot atiende peticiones, hay varias tareas ejecutándose "a la vez" dentro de ella, sin que tú hayas escrito una sola línea de código pidiéndolo. El mecanismo detrás tiene nombre: el hilo.

Proceso vs. hilo

- **Proceso:**

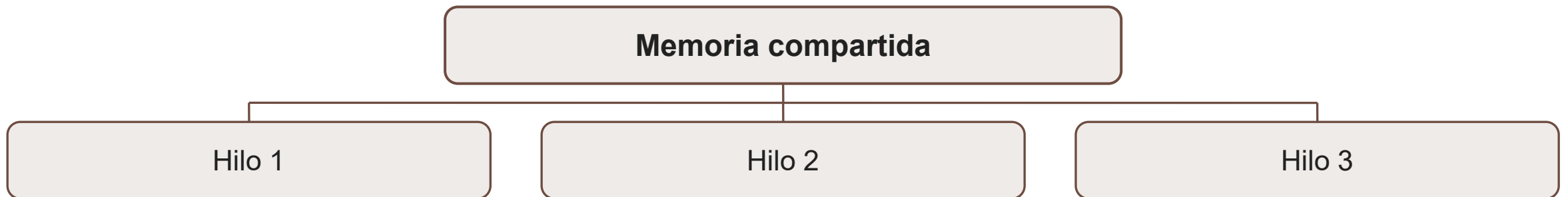
- un programa en ejecución con su propia memoria, aislada de otros procesos. La JVM entera es un proceso.

- **Hilo (thread):**

- una línea de ejecución dentro de ese proceso. Varios hilos comparten la misma memoria del proceso al que pertenecen.



Proceso (la JVM) — memoria compartida entre sus hilos



Concurrencia vs. paralelismo

Concepto	Qué significa
Concurrencia	Varios hilos se turnan para avanzar, aunque haya un solo núcleo — el planificador reparte porciones de tiempo tan rápido que parece simultáneo.
Paralelismo	Varios hilos ejecutan de verdad al mismo tiempo, en núcleos de procesador distintos.

En ambos casos, el planificador del sistema operativo decide cuándo se ejecuta cada hilo — tu programa no controla ese detalle.

Crear hilos en Java: Runnable y Thread

- **Runnable:**

- describe el trabajo a hacer — la orden de trabajo, sin decir quién la ejecuta.

- **Thread:**

- el operario de verdad — se le entrega el Runnable y, al llamar a start(), lo ejecuta de forma concurrente.

```
Runnable tarea = () -> {  
    for (int i = 0; i < 10; i++) {  
        System.out.println(Thread.currentThread().getName() + ": " + i);  
    }  
};  
  
new Thread(tarea, "Hilo-A").start();  
new Thread(tarea, "Hilo-B").start();
```

start(), no run()

Atención

Si llamas directamente a `tarea.run()`, ejecutas ese código en el hilo actual, como una llamada a método normal — no se crea ningún hilo nuevo. `start()` es lo que de verdad arranca un hilo nuevo del sistema operativo.

Ejecuta el mismo código dos veces seguidas: la salida por consola probablemente no será idéntica. El orden en que aparecen las líneas depende del planificador y no está garantizado — ambos hilos avanzan de forma concurrente, con un orden no determinista.

El problema central: compartir memoria

Condición de carrera: dos hilos modifican el mismo dato a la vez, y el resultado depende del orden exacto en que llegan a leer y escribir — un orden impredecible.

```
class Contador {  
    private int valor = 0;  
    public void incrementar() { valor++; }  
    public int getValor() { return valor; }  
}  
  
// Dos hilos llaman a incrementar() 10.000 veces cada uno
```

Cómo se pierde un incremento

valor++ no es una operación atómica: son tres pasos (leer, sumar, escribir), y se pueden intercalar entre hilos.

Paso	Hilo A	Hilo B	valor
1	lee valor → 0		0
2		lee valor → 0	0
3	calcula 0 + 1		0
4		calcula 0 + 1	0
5	escribe valor = 1		1
6		escribe valor = 1	1

Dos llamadas a incrementar() deberían dejar valor en 2, pero ambos hilos leyeron el mismo valor antes de escribir: se ha perdido un incremento entero.

La solución básica: `synchronized`

`synchronized` marca una sección crítica: solo un hilo puede ejecutarla a la vez. Cualquier otro que quiera entrar queda bloqueado por un lock, hasta que el primero sale.

```
public synchronized void incrementar() { valor++; }
```

Con `incrementar()` sincronizado, el resultado es 20000 exacto, cada vez — el `synchronized` no acelera los hilos (van más lentos, se turnan), pero garantiza que el resultado sea siempre correcto.

El peligro opuesto: deadlock

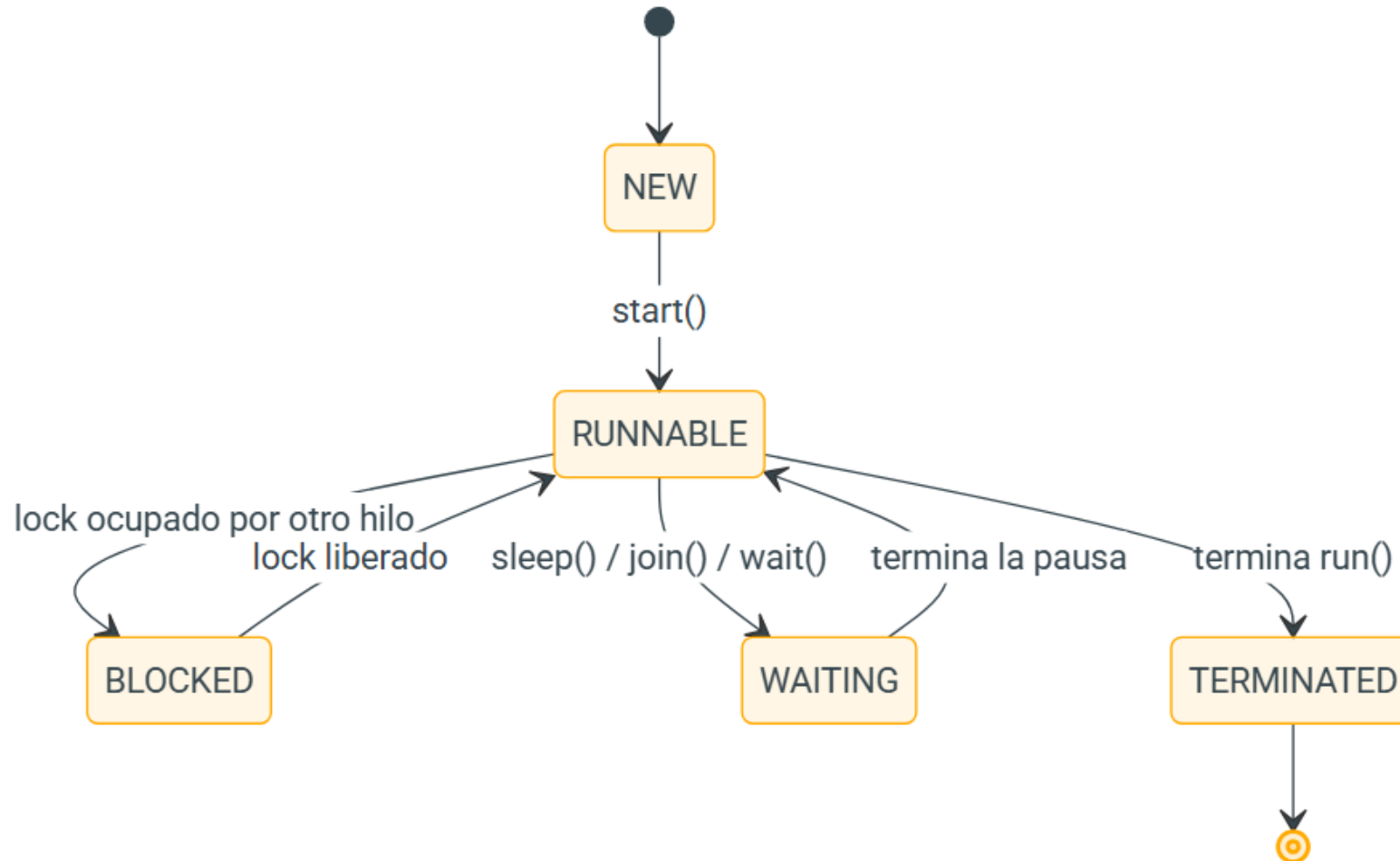
Peligro: deadlock

Si te pasas de precavido bloqueando en exceso, puedes acabar con un deadlock: dos hilos esperando cada uno un recurso que tiene bloqueado el otro, sin que ninguno pueda avanzar nunca. `synchronized` resuelve la condición de carrera, pero mal usado crea un problema distinto.

Es como el baño de una cafetería con una única llave: mientras un cliente la tiene y está dentro, cualquier otro que la necesite espera fuera — nadie entra hasta que la llave vuelve a estar libre.

El ciclo de vida de un hilo

Un hilo pasa por varios estados bien definidos entre que lo creas y termina — `Thread.getState()` permite consultar en cuál está en cada momento.



Qué significa cada estado

Estado	¿Cómo se llega?	¿Qué significa?
NEW	<code>new Thread(...)</code>	Existe el objeto, todavía no se ha llamado a <code>start()</code>
RUNNABLE	Llamas a <code>start()</code>	Ejecuta <code>run()</code> , o listo para hacerlo
BLOCKED	Intenta entrar en un <code>synchronized</code> ocupado	Espera a poder obtener el lock
WAITING	<code>join()</code> o <code>wait()</code> sin tiempo máximo	Espera sin límite
TIMED_WAITING	<code>sleep()</code> , <code>join(t)</code> o <code>wait(t)</code>	Espera durante un tiempo máximo
TERMINATED	Su <code>run()</code> acaba	Fin — no vuelve a ejecutarse

Observar los estados con getState()

```
Thread hilo1 = new Thread(contador::incrementar, "Hilo-1");
hilo1.getState();           // NEW
hilo1.start();
hilo1.getState();           // RUNNABLE
// ... compitiendo por el lock ...
hilo1.getState();           // BLOCKED
// ... dentro, durmiendo (sleep) ...
hilo1.getState();           // TIMED_WAITING
hilo1.join();
hilo1.getState();           // TERMINATED
```



Código completo (EstadosDemo): apuntes web — Tema 3, «Hilos en una aplicación real»

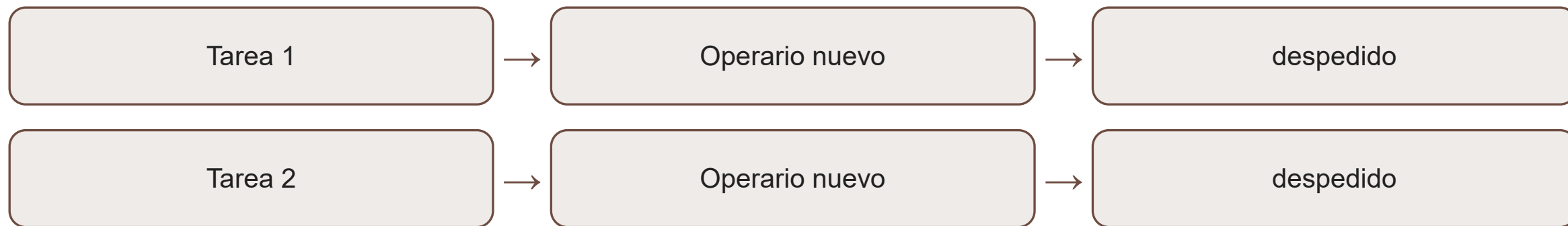
La escala de abstracción

Crear hilos con Thread/Runnable a mano no escala: un hilo nuevo por cada tarea puede agotar los recursos del sistema.

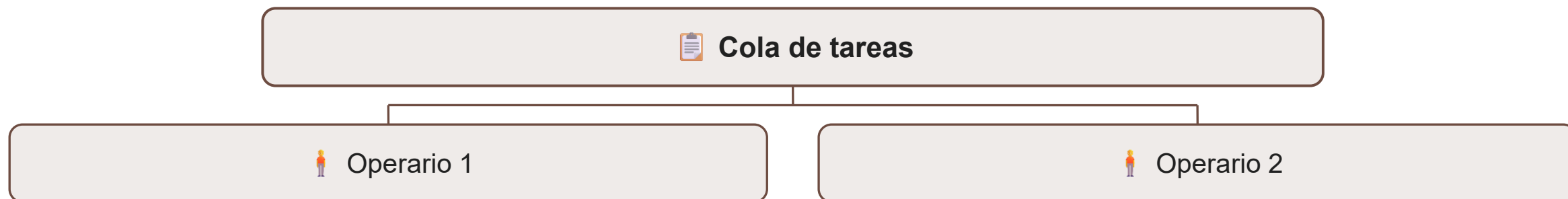
Nivel	Qué haces tú	Quién gestiona los hilos
Thread/Runnable a mano	Creas y arrancas cada hilo tú mismo	Nadie — un hilo nuevo por new Thread()
ExecutorService	Envías tareas a un pool ya creado	El propio ExecutorService, reutilizando hilos
Abstracciones de Spring	Declaras una anotación en tu código	Spring, con un ExecutorService propio

Un Thread nuevo por tarea vs. plantilla fija

 **Un Thread nuevo por tarea — se contrata y se despide en cada tarea**



 **ExecutorService — plantilla fija que se reparte la cola de tareas**



Tu aplicación ya es multihilo

El pool de Tomcat

Tomcat mantiene un pool de hilos, y cada petición HTTP se atiende en un hilo distinto de ese pool — esto ya lo tienes, sin haberlo pedido.

La próxima pieza: un listener que atiende mensajes

El pool de Tomcat funciona porque hay hilos dedicados a atender algo que llega desde fuera. Lo que vas a construir en la Actividad 3.1 sigue el mismo patrón, pero atendiendo mensajes en vez de peticiones HTTP.

El problema, antes que la solución

Registrar la actividad de un Libro (auditoría) llamando en el mismo hilo de la petición acopla dos cosas que no tienen por qué ir juntas:

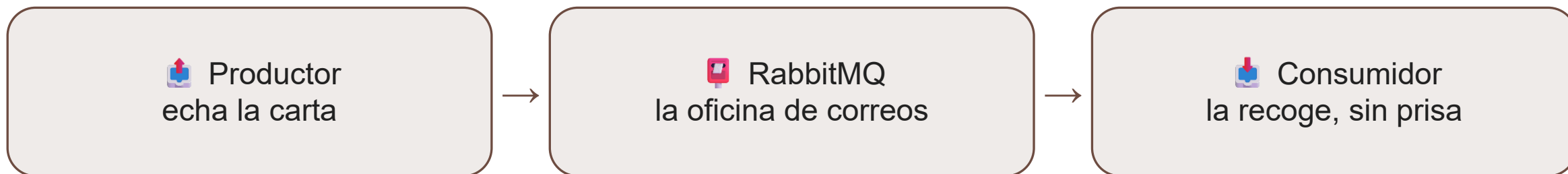
```
public LibroResponseDTO create(LibroCreateDTO dto) {  
    Libro saved = libroRepository.save(mapToEntity(dto));  
    actividadService.registrar("CREADO", "Libro",  
        saved.getId().toString()); // en el mismo hilo A  
    return mapToDTO(saved);  
}
```

El cliente que hizo el POST espera también a que termine el registro de actividad, aunque no le importe. Si registrar() fallara o tardara, se llevaría por delante la creación del libro entera.

Por qué un broker, y no simplemente otro hilo

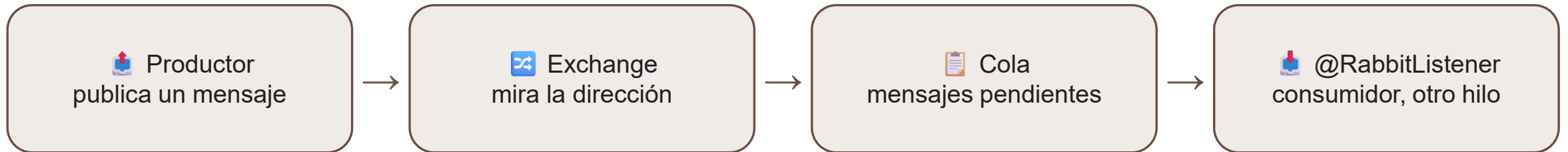
Un hilo suelto (`new Thread()`) no sobrevive a nada: si la app se reinicia justo después, el registro se pierde para siempre. Un broker de mensajería guarda el mensaje hasta que alguien lo consume de verdad.

RabbitMQ: la oficina de correos



Exchange, cola y listener

- **Cola:**
 - lista de mensajes pendientes de procesar.
- **Exchange:**
 - clasificador que mira la routing key y decide a qué cola manda cada mensaje.
- **Listener:**
 - código que reacciona cuando llega un mensaje — `@RabbitListener` lo invoca en un hilo aparte.



El binding: la ficha del clasificador

El publicador solo conoce el exchange; el listener solo conoce la cola. El binding es la regla que asocia ambos bajo un patrón de routing key.

```
@Bean
public TopicExchange catalogoExchange() {
    return new TopicExchange(CATALOGO_EXCHANGE);
}
@Bean
public Queue actividadLibroQueue() {
    return new Queue(ACTIVIDAD_LIBRO_QUEUE);
}
@Bean
public Binding actividadBinding(Queue q, TopicExchange ex) {
    return BindingBuilder.bind(q).to(ex).with("libro.*");
}
```

El mensaje y quién lo publica

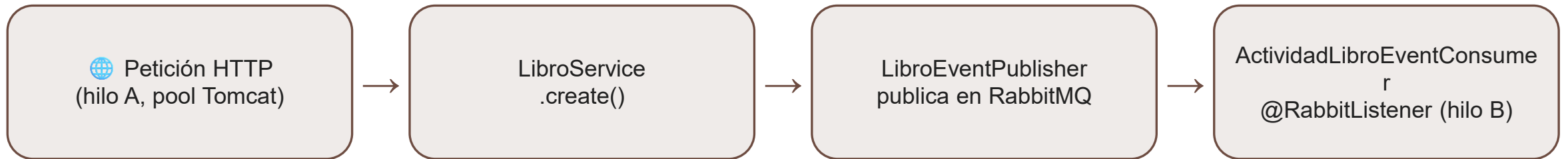
```
public record LibroEvent(String tipo, Long libroId) {}
```

```
public void publicar(String tipo, Long libroId) {  
    String payload = jsonMapper.writeValueAsString(  
        new LibroEvent(tipo, libroId));  
    rabbitTemplate.convertAndSend(CATALOGO_EXCHANGE,  
        "libro." + tipo.toLowerCase(), payload);  
}
```

`LibroService.create()` llama ahora a `publicar(...)` en vez de invocar directamente a `actividadService.registrar(...)`, y responde al cliente sin esperar.

El flujo completo: dos hilos distintos

```
@RabbitListener(queues = ACTIVIDAD_LIBRO_QUEUE)
public void recibir(String payload) {
    LibroEvent event = jsonMapper.readValue(payload, LibroEvent.class);
    actividadService.registrar(event.tipo(), "Libro",
        event.libroId().toString());
}
```



Ideas clave

- Proceso: memoria propia. Hilo: línea de ejecución dentro de un proceso, compartiendo su memoria.
- Concurrencia (turnarse) vs. paralelismo (a la vez, en núcleos distintos) — decide el planificador del SO.
- `new Thread(runnable).start()` arranca un hilo real; `run()` directamente NO crea ningún hilo nuevo.
- Condición de carrera: varios hilos modifican el mismo dato sin coordinación. `synchronized` la resuelve con un lock — en exceso, puede causar deadlock.
- Ciclo de vida: `NEW` → `RUNNABLE` → `BLOCKED` / `WAITING` / `TIMED_WAITING` → `TERMINATED`.
- RabbitMQ: broker con colas y exchanges; `@RabbitListener` procesa en un hilo aparte. El binding conecta publicador y consumidor, que no se conocen entre sí.
- Tu aplicación ya es multihilo: el pool de Tomcat ya lo tienes; el listener de RabbitMQ sigue el mismo patrón.